



Programa Certified AI-LLM Solution Architect

Curso Diseño de Infraestructura Escalable

Claves para construir plataformas eficientes y adaptables en la nube

www.bsginstitute.com

Andrés Felipe Rojas Parra

- Ingeniero electrónico con mas de 25 años de experiencia
- PGP Artificial Intelligence & Machine Learning at Texas University – Austin, TX
- Master en Inteligencia Artificial y Big Data – IEBSchool
- Estudiante de PGP en Cloud Computing at Texas University - Austin, TX
- Texas Executive Education Program – McCombs School of Business – Decision Science and AI Immersion Program
- Especialista en MLOps de la Universidad de Duke
- Estudiante de Deep Learning for Healthcare Specialization
- Director de Inteligencia Artificial e Innovacion Universidad EAN/Colombia



arojaspa@outlook.com



@arojaspa



<https://www.linkedin.com/in/arojaspa/>



+573005906373



Autoescalado + Pruebas de Carga de Agentes LLM

Capítulo 5: Técnicas de mejora de rendimiento

+ Agente LLM con patrón ReAct y pruebas de carga reales



¿Qué haremos en 3 horas?



0:00 – 0:45

5.1 Autoescalado K8s

- HPA fórmula y demo Minikube
- KEDA scale-to-zero
- AKS vs GKE vs EKS



1:30 – 2:15

5.3 Pruebas de Carga

- k6: smoke, load, stress del agente
- Observar HPA escalando en vivo
- Locust como alternativa Python



0:45 – 1:30

5.2 Agente LLM ReAct

- ¿Qué es un agente vs un chat?
- Patrón ReAct: Thought/Action/Observation
- Demo en vivo: calculadora, clima, búsqueda



2:15 – 3:00

Análisis de Resultados

- P50/P95/P99 para agentes LLM
- Benchmark multi-cloud
- ¿Cuántas réplicas necesita el agente?

HPA: la fórmula del autoescalado

$$\text{replicas_deseadas} = \text{ceil}(\text{replicas_actuales} \times \text{métr_actual} \div \text{objetivo})$$

Agente: 2 pods × (75% CPU ÷ 60%) = ceil(2.5) = 3 pods | API simple: threshold 70% CPU

HPA

Horizontal Pod
Autoscaler

¿Qué? Escala RÉPLICAS del pod
según CPU, RAM o métrica
custom

¿Cuándo? Siempre — primera opción
para cualquier workload

⌚ Tiempo ~30 seg

KEDA

Event-Driven
Autoscaling

¿Qué? Extiende HPA: escala a 0
Triggers: HTTP, Redis, Kafka...

¿Cuándo? Tráfico variable o
horario definido

⌚ Tiempo ~30 seg (0→N)



CAS/Karpenter

Cluster
Autoscaler

¿Qué? Escala NODOS (VMs)
cuando los pods no caben

¿Cuándo? Automático tras el HPA
cuando nodos llenos

⌚ Tiempo 2-10 min

Autoescalado en Azure / GCP / AWS

Característica	 Azure AKS	 GCP GKE	 AWS EKS
HPA nativo	✓ Nativo	✓ Nativo	✓ Nativo
KEDA	✓ Addon oficial	Community	Community
Cluster Autoscaler	✓ Integrado	Autopilot(auto)	Karpenter <2min
Scale-to-zero	KEDA HTTP	KEDA / Autopilot	Karpenter
Pago por nodo	Solo activos	Por pod (Autopilot)	Spot: -90%



¿Cuál elegir para un Agente LLM?



Azure AKS + KEDA:

Mejor si ya usas Azure OpenAI. KEDA nativo, integración perfecta.



GKE Autopilot:

Más fácil de operar. Pagas por pod, no por nodo. Ideal para demos.



EKS + Karpenter:

Más barato con Spot instances. Karpenter escala nodos en <2 min.

Chat vs Agente: la diferencia clave



Chat simple

Usuario: «¿Cuánto es 1234×5678 ?»

LLM: « $1234 \times 5678 = 7,006,672$ »

⚠ **INCORRECTO** (los LLMs

cometen errores matemáticos)

— FIN —

1 LLM call

Respuesta directa



Agente LLM (ReAct)

Usuario: «¿Cuánto es 1234×5678 ?»

Thought: necesito calcular...

Action: calculator

Observation: 7,006,652

Final Answer: $1234 \times 5678 = 7,006,652$ ✓

3 LLM calls

Resultado exacto ✓

ReAct: Reasoning + Acting

1 →

Thought

El LLM razona
qué hacer

2 →

Action

Decide qué
herramienta usar

3 →

Action Input

El argumento
para la herramienta

4

Observation

Resultado de
la herramienta

✓ Final Answer

Si necesita más información → repite el loop (máx. MAX_STEPS)

Ejemplo: «¿Cuánto es $365 \times 24 \times 60$ y qué clima hay en Lima?»

Thought: necesito calcular. Action: calculator | Input: $365 \times 24 \times 60$ → Obs: 525,600

Thought: ahora el clima. Action: weather | Input: Lima → Obs: Lima 18°C nublado

Final Answer: $365 \times 24 \times 60 = 525,600$ minutos. En Lima hay 18°C con cielo nublado.

Las 3 herramientas: arquitectura y seguridad

calculator

Cálculos matemáticos seguros

⚙️ Cómo funciona

ast.parse() + whitelist
de funciones permitidas

🔒 Seguridad

eval() bloqueado
Solo math.sqrt, sin, cos...

💡 Por qué

Los LLMs se equivocan
en operaciones exactas

```
Action: calculator
Input: 1234 * 5678
Obs: 7,006,652
```

weather

Clima de ciudades (mock→API real)

⚙️ Cómo funciona

Mock con asyncio.sleep()
Simula latencia API real

🔒 Seguridad

En prod: API key en
Azure Key Vault / Secrets

💡 Por qué

Reemplazar _fetch_real()
sin tocar el agente

```
Action: weather
Input: Bogotá
Obs: 14°C parcialmente nublado
```

search

Búsqueda info técnica (mock→Tavily)

⚙️ Cómo funciona

Base de conocimiento
del curso en dict Python

🔒 Seguridad

En prod: Tavily API
(optimizado para LLMs)

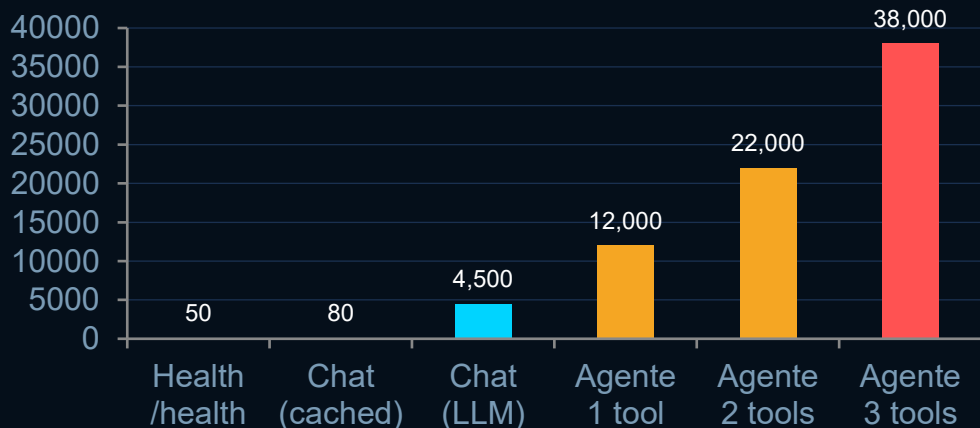
💡 Por qué

Mismo código, distinto
backend según entorno

```
Action: search
Input: ¿qué es KEDA?
Obs: KEDA extiende HPA...
```

El agente: latencia, CPU y memoria

Latencia según tipo de request



Implicaciones para el HPA

- CPU threshold más bajo**
60% (vs 70% para APIs)
el agente consume más CPU
por múltiples LLM calls
- Timeouts largos**
k6 timeout: 60-90s
no 10s como para
APIs REST simples
- Memoria por sesión**
Historial en RAM → Redis
para que HPA escale
sin perder contexto

Costo CPU por paso ReAct:

Thought (LLM):

~0.4 CPU-seg

Action (herramienta):

<0.01 CPU-seg

Observation (parsing):

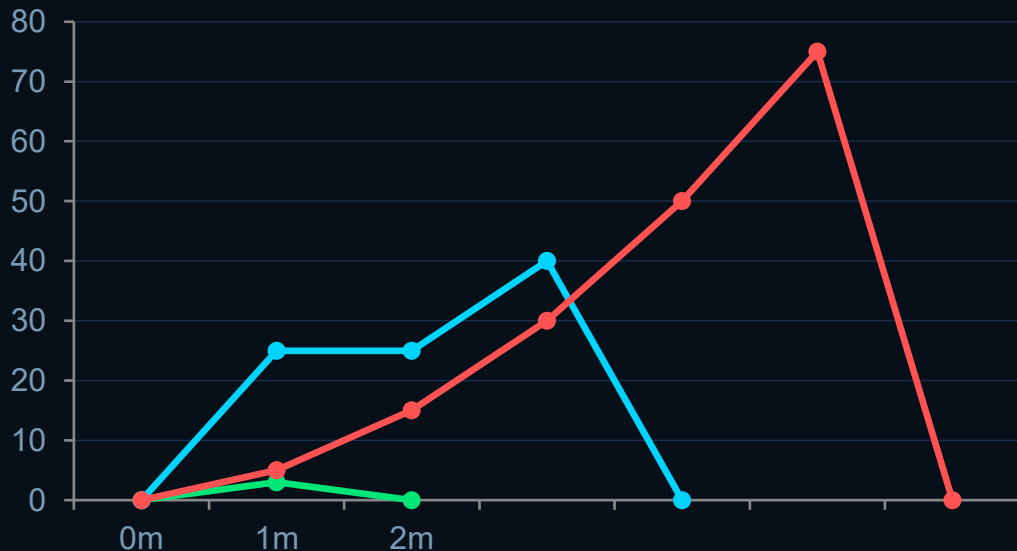
<0.01 CPU-seg

Total (3 pasos):

~1.2 CPU-seg/req

Tipos de prueba para el Agente LLM

Usuarios virtuales vs tiempo (agente LLM)



Smoke

3 VUs, 2 min · P95 < 45s, errores 0%

Agente necesita VUs bajos y timeout largo



Load

25→40 VUs, 10m · P95 < 45s, errores < 10%

Mix: 35% health, 35% chat, 20% agente simple, 10% complejo



Stress

hasta 75 VUs, 10m · P99 < 120s, errores < 40%

503 (circuit breaker) y 429 (rate limit) son ESPERADOS bajo estrés

⚠ Diferencia clave vs APIs REST: el agente necesita timeouts de 60-90s, no 10s

Demo: k6 + HPA del agente en Minikube

Comandos — 3 terminales

Terminal 1: Setup K8s

```
minikube start --cpus=4 --memory=8192
minikube addons enable metrics-server
kubectl apply -f kubernetes/local/
```

Terminal 2: Monitorear HPA

```
watch -n 2 kubectl get hpa,pods -n llm-prod
```

Terminal 3: Load test del agente

```
k6 run loadtesting/k6/load-test.js
```

Extra: probar el agente manualmente

```
curl -X POST localhost:8000/agent/run \
  -d '{"input": "¿Cuánto es 999*888?"}'
```

Output HPA esperado

--- Sin carga ---

TARGETS	REPLICAS
12%/60%	2

--- k6 ramp-up ---

68%/60% → **escala** ⬆

68%/60% 4 ← pods!

--- Carga sostenida ---

55%/60% 4 estable

--- k6 termina ---

12%/60% 2 ← baja

(tras 5 min cooldown)

Cómo interpretar resultados del agente LLM

Output k6 — Agente LLM

http_reqs.....: 412 0.69/s

http_req_failed.....: 3.4% ✓ <10%

agent_latency_ms.....:

avg=8.3s p(50)=5.1s

p(90)=18.4s p(95)=26.8s

p(99)=41.2s max=58.3s

chat_latency_ms p(95)=3.2s

cache_hits_count: 187 (45%)

agent_tool_calls: 284

agent_steps_avg: 2.1 pasos

✓ RESULTADO: PASS

P50 = 5.1s

La mitad de ejecuciones del agente terminan en 5.1s. Normal para 1-2 herramientas con Ollama en CPU.

P95 = 26.8s

El 5% más lento. Queries con 3+ herramientas o LLM bajo carga. Añadir réplicas baja este número.

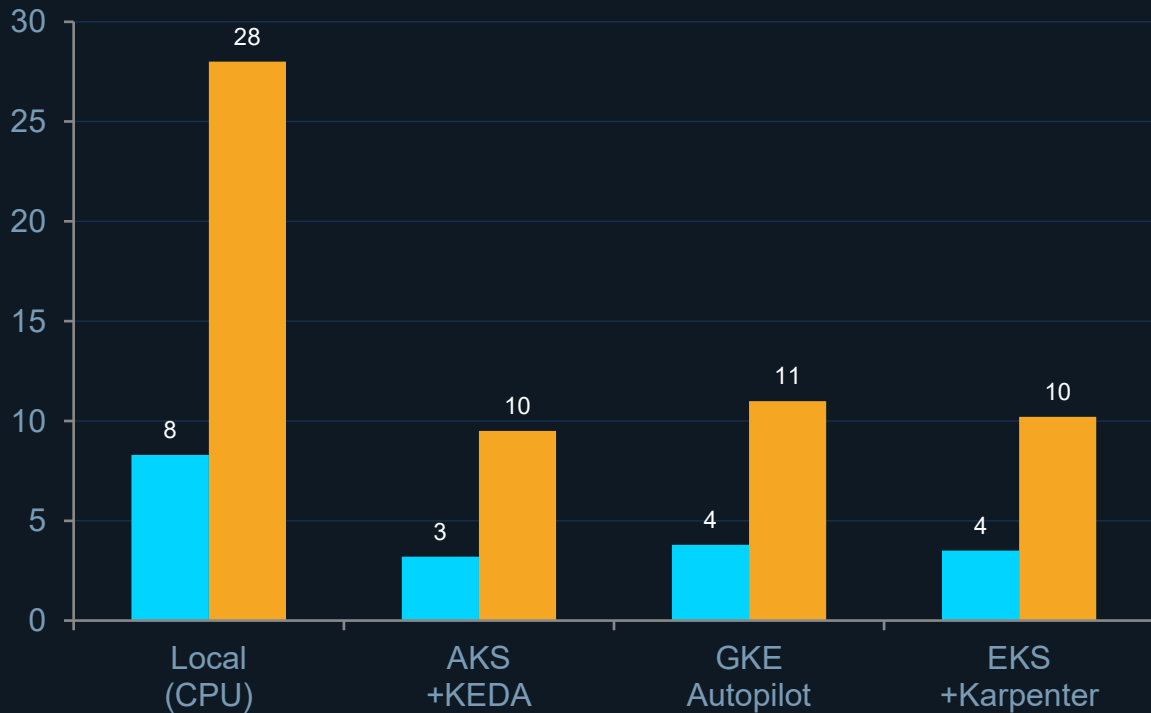
Cache = 45%

Casi la mitad de requests sirvió desde Redis (<80ms). Más usuarios → más cache hits → mejor P95.

Avg 2.1 pasos

Promedio de pasos ReAct por ejecución. Queries simples: 1-2 pasos. Complejas: 3-5 pasos.

Benchmark del Agente: Local vs Azure vs GCP vs AWS



* Valores de referencia con llama3.2:3b. Con gpt-4o/Claude via API: P95 ~3-5s independiente del cloud



K8s en nube

P95 ~ 9-11s vs 28s local. El HPA mantiene pods calientes. Ollama en RAM siempre disponible.



AKS + KEDA

Mejor P95 (9.5s). KEDA HTTP scaler reduce latencia ante picos preparando pods extra.



Local CPU

P50 = 8.3s, P95 = 28s. llama3.2:3b en CPU: 10-20 tok/s. Con GPU: 3-5x más rápido.

Prometheus + Grafana para el Agente LLM

agent_runs_total

Counter

Total ejecuciones del agente por éxito y sesión

→ Alertar si tasa de errores > 10%

agent_tool_calls_total

Counter

Llamadas por herramienta (calculator, weather...)

→ Identificar herramienta más usada

agent_duration_seconds

Histogram

Latencia del agente en segundos (P50/P95/P99)

→ Dashboard principal de SLA

llm_active_connections

Gauge

Conexiones activas al momento

→ Trigger para KEDA ScaledObject

agent_steps_total

Histogram

Pasos de razonamiento por ejecución

→ Detectar queries que generan loops

llm_cache_hits_total

Counter

Hits en cache Redis

→ Medir efectividad del cache

k6 vs Locust: ¿cuál usar?

k6 (JavaScript)

✓ Ventajas:

- Sintaxis clara y concisa
- Métricas integradas (P95, P99)
- Output HTML/JSON/CSV
- Ejecuta en CI/CD fácilmente
- Thresholds automáticos

⚠ Limitaciones:

- Requiere aprender JS
- Menos Pythónico para data scientists

```
k6 run load-test.js
```

Locust (Python)

✓ Ventajas:

- 100% Python — familiar para DS
- UI web en `http://localhost:8089`
- Fácil extensión con clases
- Ideal para lógica compleja
- Resultados CSV y gráficas web

⚠ Limitaciones:

- Más código para lograr lo mismo
- Sin thresholds automáticos nativos

```
locust -f locustfile.py  
--host http://localhost:8000
```

SESIÓN 7 COMPLETADA ✓

Lo que dominaste hoy



HPA y KEDA en K8s local y en Azure AKS / GKE Autopilot / EKS+Karpenter



Agente LLM con patrón ReAct: herramientas, prompts, memoria de sesión



Herramientas seguras: calculadora con AST whitelist, clima y búsqueda mock



k6: smoke, load y stress test del agente con métricas P50/P95/P99



Benchmark multi-cloud: por qué el agente en K8s es 3x más rápido que local

Próxima sesión (Sesión 8):

Seguridad y Ética en Soluciones LLM — Guardrails, PII detection, prompt injection, cumplimiento normativo